

# Machine Intelligence

## 10. Classical Planning

Automated Sequential Decision Making on “Simple” Environments

Álvaro Torralba



AALBORG UNIVERSITET

Fall 2024

# Planning

## Ambition:

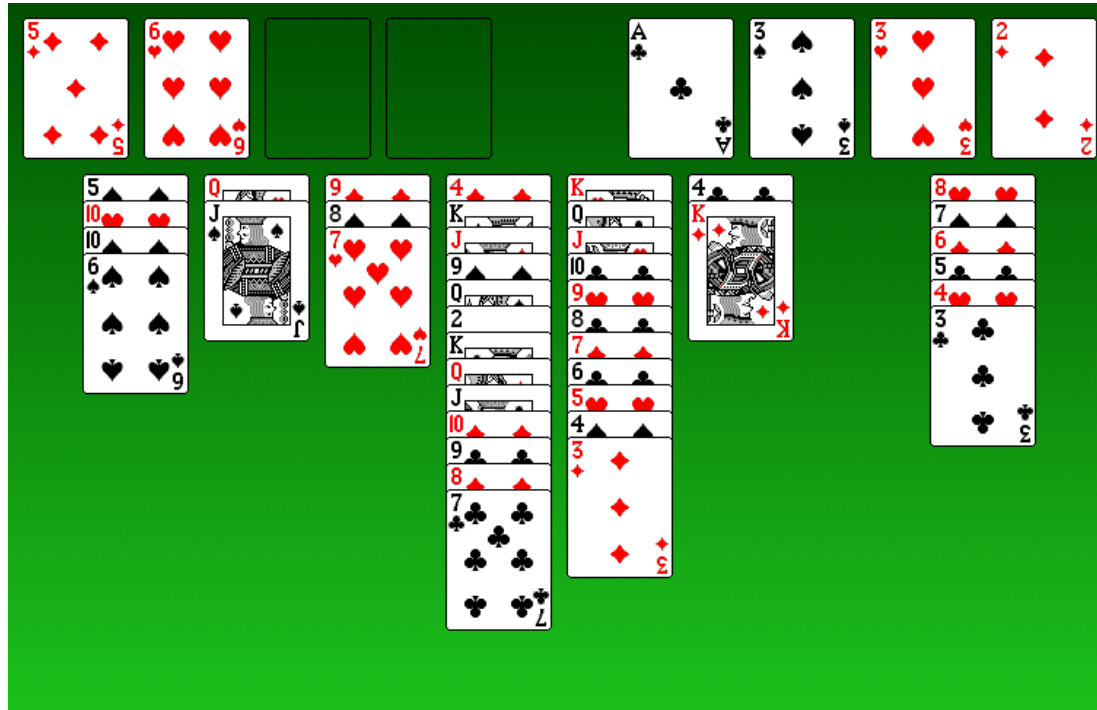
**Write one program (planner) that can solve all sequential decision-making problems.**

## How do we describe our problem to the planner?

- A *logical description* of the possible **states**
- A *logical description* of the **initial state**  $I$
- A *logical description* of the **goal condition**  $G$
- *logical description* of the set  $A$  of **actions** in terms of **preconditions** and **effects**

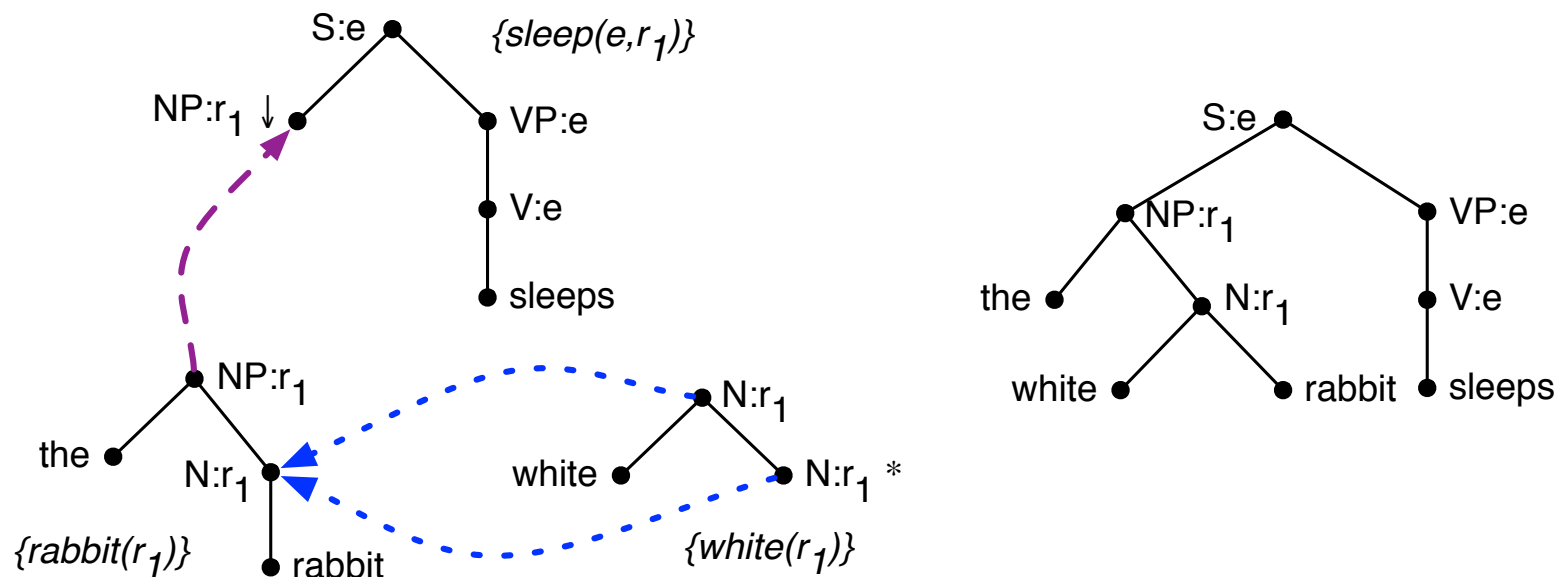
→ Solution (**plan**) = **sequence of actions** from  $A$ , transforming  $I$  into a state that satisfies  $G$ .

## Example of a Planning Task



- **States:** Card positions ( $position\_Jspades = Qhearts$ ).
- **Actions:** Card moves ( $move\_Jspades\_Qhearts\_freecell4$ ).
- **Initial state:** Start configuration.
- **Goal states:** All cards “home”.
- **Solution:** Card moves solving this game.

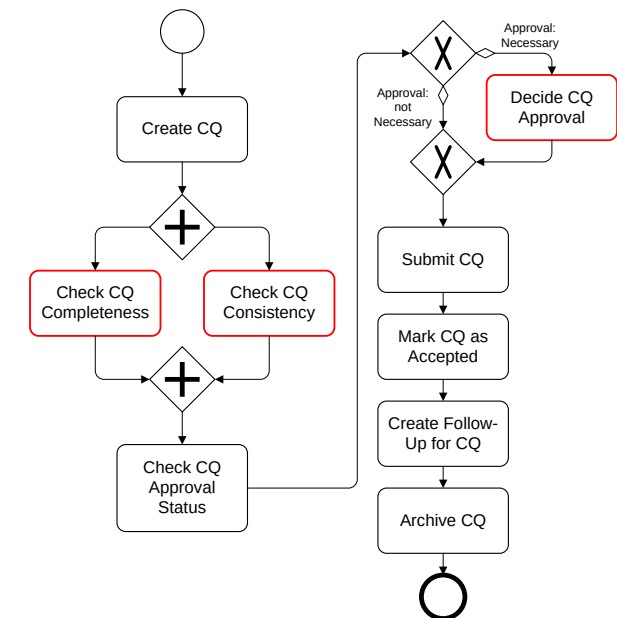
# Planning: Language Generation



- **Input:** Tree-adjoining grammar, intended meaning.
- **Output:** Sentence expressing that meaning.

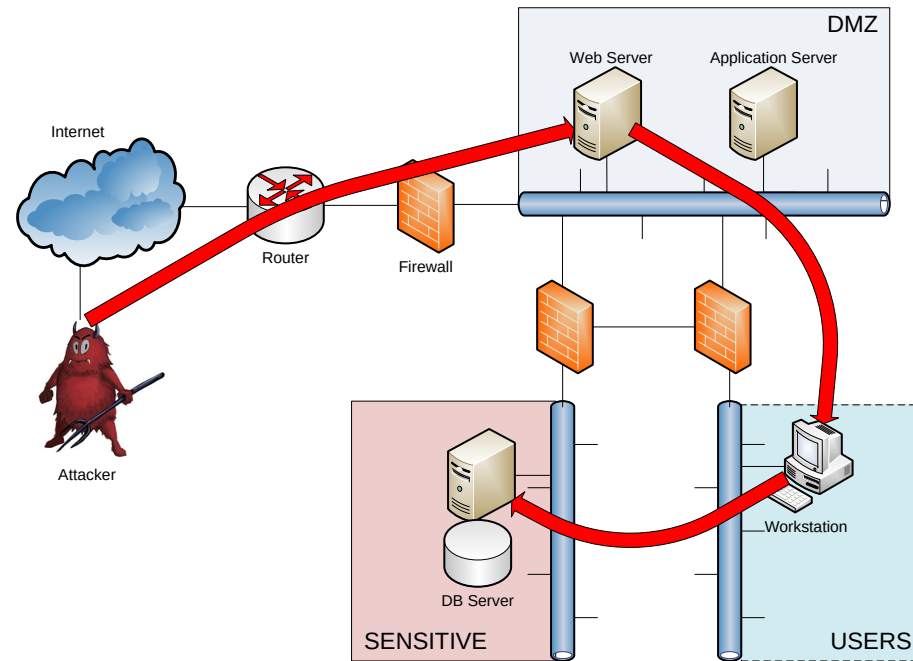
# Planning: Business Process Templates at SAP

| Action name              | precondition                                                                                                            | effect                                                       |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| Check CQ Completeness    | CQ.archiving:notArchived                                                                                                | CQ.completeness:complete OR<br>CQ.completeness:notComplete   |
| Check CQ Consistency     | CQ.archiving:notArchived                                                                                                | CQ.consistency:consistent OR<br>CQ.consistency:notConsistent |
| Check CQ Approval Status | CQ.archiving:notArchived AND<br>CQ.approval:notChecked AND<br>CQ.completeness:complete AND<br>CQ.consistency:consistent | CQ.approval:necessary OR<br>CQ.approval:notNecessary         |
| Decide CQ Approval       | CQ.archiving:notArchived AND<br>CQ.approval:necessary                                                                   | CQ.approval:granted OR<br>CQ.approval:notGranted             |
| Submit CQ                | CQ.archiving:notArchived AND<br>(CQ.approval:notNecessary OR<br>CQ.approval:granted)                                    | CQ.submission:submitted                                      |
| Mark CQ as Accepted      | CQ.archiving:notArchived AND<br>CQ.submission:submitted                                                                 | CQ.acceptance:accepted                                       |
| Create Follow-Up for CQ  | CQ.archiving:notArchived AND<br>CQ.acceptance:accepted                                                                  | CQ.followUp:documentCreated                                  |
| Archive CQ               | CQ.archiving:notArchived                                                                                                | CQ.archiving:archived                                        |



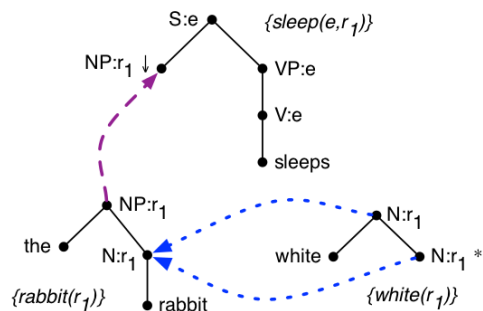
- **Input:** SAP-scale model of behavior of activities on Business Objects, process endpoint.
- **Output:** Process template leading to this point.

# Planning: Security Testing

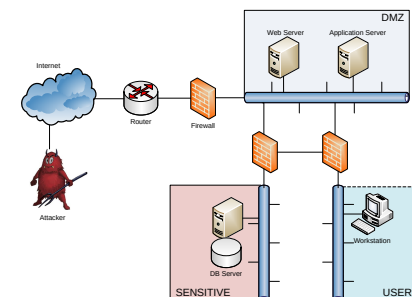


- **Input:** Network configuration, location of sensible data.
- **Output:** Sequence of exploits giving access to that data.

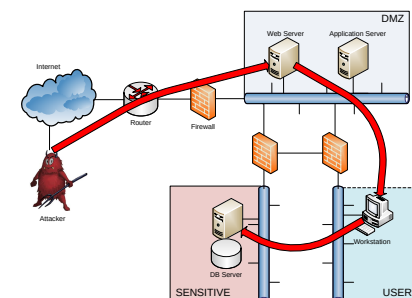
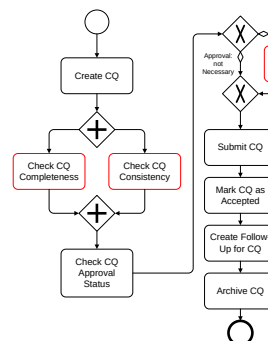
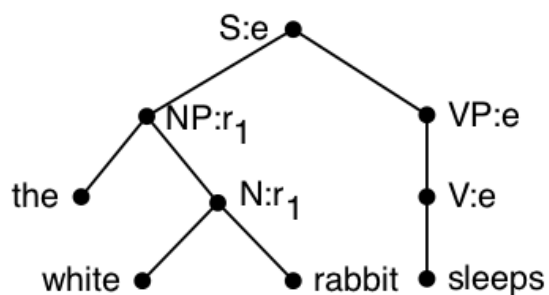
# Planning!



| Action name              | precondition                                                                                                   | effect                                                    |
|--------------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Check CQ Completeness    | CQ.archiving:notArchived                                                                                       | CQ.completeness:complete OR CQ.completeness:notComplete   |
| Check CQ Consistency     | CQ.archiving:notArchived                                                                                       | CQ.consistency:consistent OR CQ.consistency:notConsistent |
| Check CQ Approval Status | CQ.archiving:notArchived AND CQ.approval:notChecked AND CQ.completeness:complete AND CQ.consistency:consistent | CQ.approval:necessary OR CQ.approval:notNecessary         |
| Decide CQ Approval       | CQ.archiving:notArchived AND CQ.approval:necessary                                                             | CQ.approval:granted OR CQ.approval:notGranted             |
| Submit CQ                | CQ.archiving:notArchived AND (CQ.approval:notNecessary OR CQ.approval:granted)                                 | CQ.submission:submitted                                   |
| Mark CQ as Accepted      | CQ.archiving:notArchived AND CQ.submission:submitted                                                           | CQ.acceptance:accepted                                    |
| Create Follow-Up for CQ  | CQ.archiving:notArchived AND CQ.acceptance:accepted                                                            | CQ.followUp:documentCreated                               |
| Archive CQ               | CQ.archiving:notArchived                                                                                       | CQ.archiving:archived                                     |



Planning Domain Definition Language (PDDL)  $\mapsto$  Planning System



# Algorithmic Problems in Planning

## Satisficing Planning

**Input:** A planning task  $\Pi$ .

**Output:** A plan for  $\Pi$ , or “unsolvable” if no plan for  $\Pi$  exists.

## Optimal Planning

**Input:** A planning task  $\Pi$ .

**Output:** An *optimal* plan for  $\Pi$ , or “unsolvable” if no plan for  $\Pi$  exists.

→ The techniques successful for either one of these are almost disjoint. And satisficing planning is much more effective in practice.

→ Programs solving these problems are called (optimal) **planners**, **planning systems**, or **planning tools**.



## Our Agenda for This Chapter

- **The STRIPS Planning Formalism:** How do we represent a planning task?  
→ **Lays the framework we'll be looking at.**
- **Planning Domain Definition Language:** How we actually express our classical planning problems.  
→ **For reference.**
- **Planning as Heuristic Search:** How state-space search techniques are applied to solve planning tasks?  
→ **A Recap of Chapter 9.**
- **The Delete Relaxation:** How to relax a planning problem?  
→ **The delete relaxation is the most successful method for the automatic generation of heuristic functions. It relaxes STRIPS planning tasks by ignoring the delete lists.**
- **The  $h^+$  Heuristic:** What is the resulting heuristic function?  
→  **$h^+$  is the “ideal” delete relaxation heuristic.**
- **Approximating  $h^+$ :** How to actually compute a heuristic?  
→ **Turns out that, in practice, we must approximate  $h^+$ .**

# “STRIPS” Planning

- **STRIPS** = Stanford Research Institute Problem Solver.  
*STRIPS is the simplest possible (reasonably expressive) logics-based planning language.*
- STRIPS has only **Boolean variables**: propositional logic atoms.
- Its preconditions/effects/goals are as canonical as imaginable:
  - Preconditions, goals: **conjunctions** of **positive atoms**.
  - Effects: **conjunctions** of **literals** (positive or negated atoms).
- We use the common set-based notation for this simple formalism.

→ Historical note: STRIPS [?] was originally a planner, whose language actually wasn't quite that simple.

# STRIPS Planning: Syntax

**Definition (STRIPS Planning Task).** A **STRIPS planning task**, short **planning task**, is a 4-tuple  $\Pi = (P, A, I, G)$  where:

- $P$  is a finite set of **facts** (aka **propositions**).
- $A$  is a finite set of **actions**; each  $a \in A$  is a triple  $a = (pre_a, add_a, del_a)$  of subsets of  $P$  referred to as the action's **precondition**, **add list**, and **delete list** respectively; we require that  $add_a \cap del_a = \emptyset$ .
- $I \subseteq P$  is the **initial state**.
- $G \subseteq P$  is the **goal**.

We will often give each action  $a \in A$  a **name** (a string), and identify  $a$  with that name.

**Note:** We assume **unit costs** for simplicity: every action has cost 1.

# “TSP” in Australia



# STRIPS Encoding of “TSP”



- **Facts**  $P$ :  $\{at(x), visited(x) \mid x \in \{Sydney, Adelaide, Brisbane, Perth, Darwin\}\}$ .
- **Initial state**  $I$ :  $\{at(Sydney), visited(Sydney)\}$ .
- **Goal**  $G$ :  $\{at(Sydney)\} \cup \{visited(x) \mid x \in \{Sydney, Adelaide, Brisbane, Perth, Darwin\}\}$ .
- **Actions**  $a \in A$ :  $drive(x, y)$  where  $x, y$  have a road.  
**Precondition**  $pre_a$ :  $\{at(x)\}$ .  
**Add list**  $add_a$ :  $\{at(y), visited(y)\}$ .  
**Delete list**  $del_a$ :  $\{at(x)\}$ .
- **Plan**:  $\langle drive(Sydney, Brisbane), drive(Brisbane, Sydney), drive(Sydney, Adelaide), drive(Adelaide, Perth), drive(Perth, Adelaide), drive(Adelaide, Darwin), drive(Darwin, Adelaide), drive(Adelaide, Sydney) \rangle$ .

# STRIPS Planning: Semantics

**Definition (STRIPS State Space).** Let  $\Pi = (P, A, c, I, G)$  be a STRIPS planning task. The **state space** of  $\Pi$  is  $\Theta_\Pi = (S, A, T, I, S^G)$  where:

- The states (also **world states**)  $S = 2^P$  are the subsets of  $P$ .
- $A$  is  $\Pi$ 's action set.
- The transitions are  $T = \{s \xrightarrow{a} s' \mid \text{pre}_a \subseteq s, s' = \text{appl}(s, a)\}$ .  
If  $\text{pre}_a \subseteq s$ , then  $a$  is **applicable** in  $s$  and  $\text{appl}(s, a) := (s \cup \text{add}_a) \setminus \text{del}_a$ . If  $\text{pre}_a \not\subseteq s$ , then  $\text{appl}(s, a)$  is undefined.
- $I$  is  $\Pi$ 's initial state.
- The goal states  $S^G = \{s \in S \mid G \subseteq s\}$  are those that satisfy  $\Pi$ 's goal.

An (optimal) **plan** for  $s \in S$  is an (optimal) solution for  $s$  in  $\Theta_\Pi$ , i.e., a path from  $s$  to some  $s' \in S^G$ . A solution for  $I$  is called a **plan for  $\Pi$** .  $\Pi$  is **solvable** if a plan for  $\Pi$  exists.

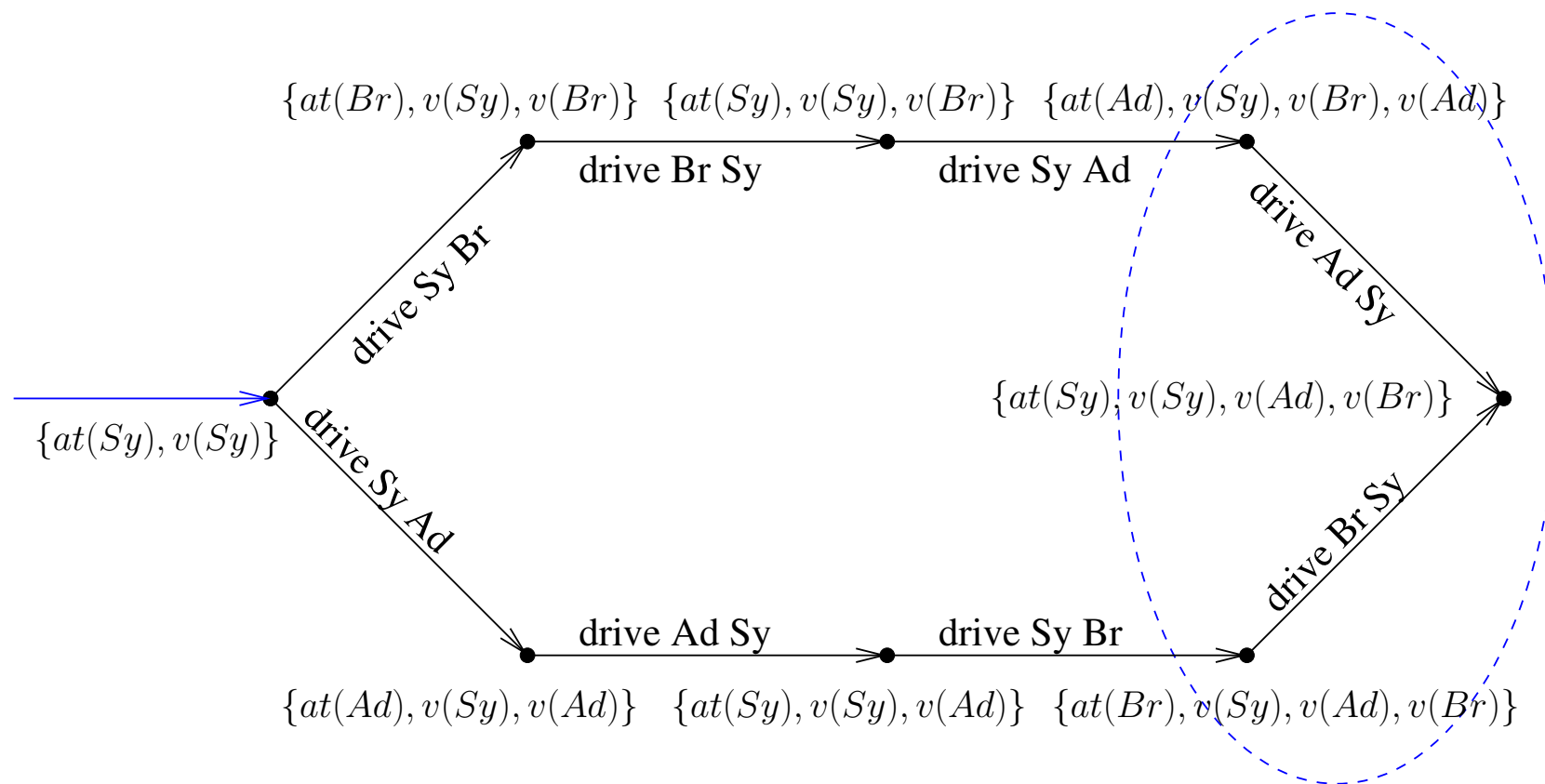
For  $\vec{a} = \langle a_1, \dots, a_n \rangle$ ,  $\text{appl}(s, \vec{a}) := \text{appl}(\dots \text{appl}(\text{appl}(s, a_1), a_2) \dots, a_n)$  if each  $a_i$  is applicable in the respective state; else,  $\text{appl}(s, \vec{a})$  is undefined.

# STRIPS Encoding of Simplified “TSP”



- **Facts**  $P$ :  $\{at(x), visited(x) \mid x \in \{Sydney, Adelaide, Brisbane\}\}$ .
- **Initial state**  $I$ :  $\{at(Sydney), visited(Sydney)\}$ .
- **Goal**  $G$ :  $\{visited(x) \mid x \in \{Sydney, Adelaide, Brisbane\}\}$ . (Note: no “ $at(Sydney)$ ”.)
- **Actions**  $a \in A$ :  $drive(x, y)$  where  $x, y$  have a road.
  - Precondition**  $pre_a$ :  $\{at(x)\}$ .
  - Add list**  $add_a$ :  $\{at(y), visited(y)\}$ .
  - Delete list**  $del_a$ :  $\{at(x)\}$ .

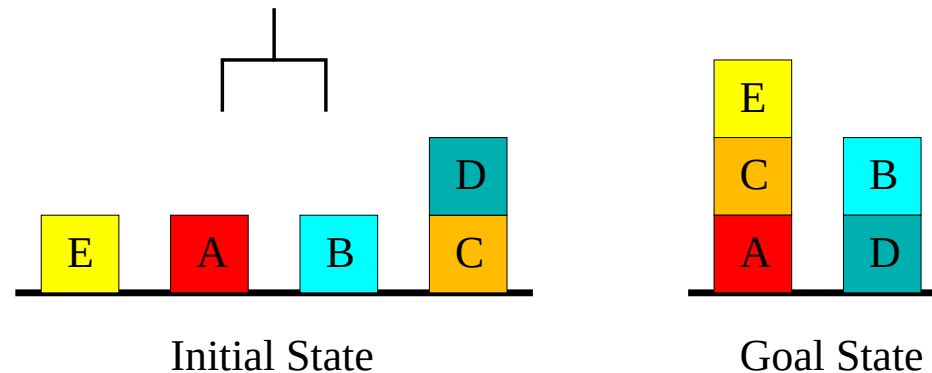
# STRIPS Encoding of Simplified “TSP”: State Space



→ **Is this actually the state space?** No, only the reachable part. E.g.,  $\Theta_{\Pi}$  also includes the states  $\{v(Sy)\}$  and  $\{at(Sy), at(Br)\}$ .



# (Oh no it's) The Blockworld



- **Facts:**  $on(x, y)$ ,  $onTable(x)$ ,  $clear(x)$ ,  $holding(x)$ ,  $armEmpty()$ .
- **Initial state:**  $\{onTable(E), clear(E), \dots, onTable(C), on(D, C), clear(D), armEmpty()\}$ .
- **Goal:**  $\{on(E, C), on(C, A), on(B, D)\}$ .
- **Actions:**  $stack(x, y)$ ,  $unstack(x, y)$ ,  $putdown(x)$ ,  $pickup(x)$ .
- **$stack(x, y)?$**   $pre : \{holding(x), clear(y)\}$   
 $add : \{on(x, y), armEmpty()\}$   
 $del : \{holding(x), clear(y)\}$ .

# Questionnaire

## Question!

Which are correct encodings (part of some correct overall encoding) of the STRIPS Blocksworld *pickup*(*x*) action schema?

(A): (*onTable*(*x*), *clear*(*x*),  
*armEmpty*()}, {*holding*(*x*)},  
{*onTable*(*x*)}).

(C): (*onTable*(*x*), *clear*(*x*),  
*armEmpty*()}, {*holding*(*x*)},  
{*onTable*(*x*), *armEmpty*()},  
*clear*(*x*)}).

(B): (*onTable*(*x*), *clear*(*x*),  
*armEmpty*()}, {*holding*(*x*)},  
{*armEmpty*()}).

(D): (*onTable*(*x*), *clear*(*x*),  
*armEmpty*()}, {*holding*(*x*)},  
{*onTable*(*x*), *armEmpty*()}).

→ (A): No, must delete *armEmpty*(). (B): No, must delete *onTable*(*x*). (C), (D): Both yes: We can, but don't have to, encode the *single-arm* Blocksworld so that the block currently in the hand is not clear. (For (C), *stack*(*x*, *y*) and *putdown*(*x*) need to add *clear*(*x*), so the encoding on the previous slide does not work.)

# Levels of Representation (Chapter 1)

We consider 3 representation schemes

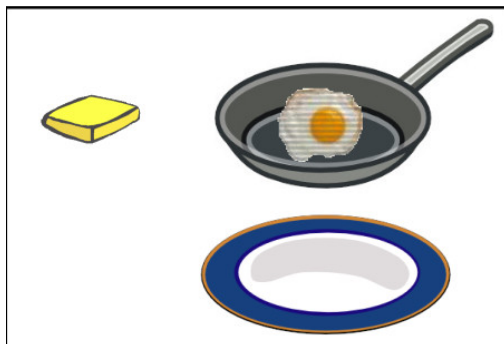
## State based:

State-space Search

(Chapter 9)



State 01



State 12

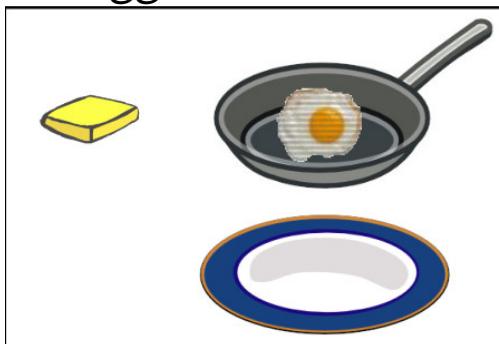
## Feature based:

STRIPS

(This Chapter)



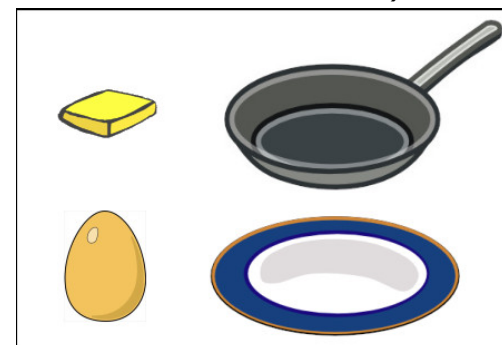
*egg=whole,  
butter\_in=table,  
egg\_in=table*



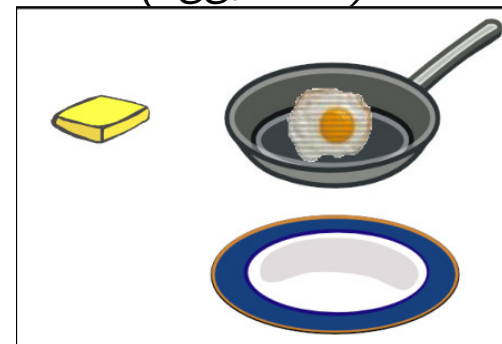
*egg=broken,  
butter\_in=table,  
egg\_in=pan*

## Relational: PDDL

(What we actually use,  
not in this course)

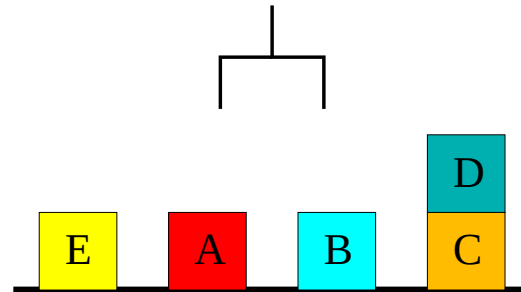


*state(egg,whole),  
in(butter,table),  
in(egg,table)*

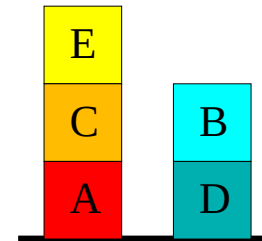


*state(egg,broken),  
in(butter,table),  
in(egg,pan)*

# The Blockworld in PDDL (STRIPS):



Initial State



Goal State

## Domain File:

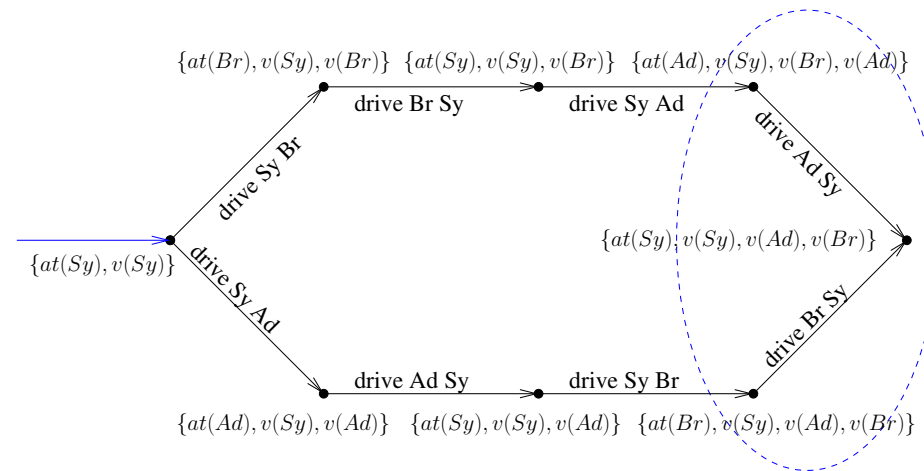
```
(define (domain blockworld)
  (:predicates (clear ?x) (holding ?x) (on ?x ?y)
               (on-table ?x) (arm-empty))
  (:action stack
    :parameters (?x ?y)
    :precondition (and (clear ?y) (holding ?x))
    :effect (and (arm-empty) (on ?x ?y)
                 (not (clear ?y)) (not (holding ?x))))
)
```

...

## Problem File:

```
(define (problem bw-abcde)
  (:domain blockworld)
  (:objects a b c d e)
  (:init (on-table a) (clear a)
         (on-table b) (clear b)
         (on-table e) (clear e)
         (on-table c) (on d c) (clear d)
         (arm-empty))
  (:goal (and (on e c) (on c a) (on b d))))
```

# So, How to Solve Planning Problems?

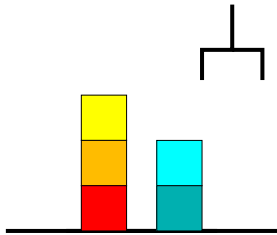


## Question

Given a graph  $G$ , a node  $I$ , and a set of nodes  $G$ , find the shortest path from  $I$  to any node in  $G$ . What algorithm do you suggest to use?

Some examples: Dijkstra,  $A^*$ , Greedy Best First Search (**Chapter 9**)

# The Problem



- $n$  blocks, 1 hand.
- A single action either takes a block with the hand or puts a block we're holding onto some other block/the table.

| blocks | states | blocks | states           |
|--------|--------|--------|------------------|
| 1      | 1      | 9      | 4596553          |
| 2      | 3      | 10     | 58941091         |
| 3      | 13     | 11     | 824073141        |
| 4      | 73     | 12     | 12470162233      |
| 5      | 501    | 13     | 202976401213     |
| 6      | 4051   | 14     | 3535017524403    |
| 7      | 37633  | 15     | 65573803186921   |
| 8      | 394353 | 16     | 1290434218669921 |

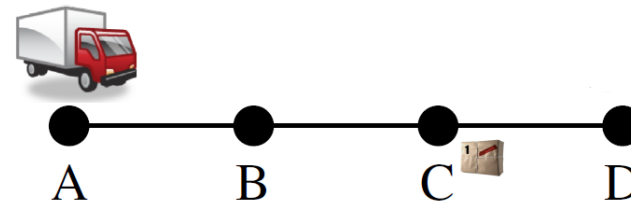
State Explosion Problem:

→ State spaces typically are huge (not only for Go and Chess).

In practice, the graph is generated on the fly!

## Example: Planning as Search

### Example: “Logistics”

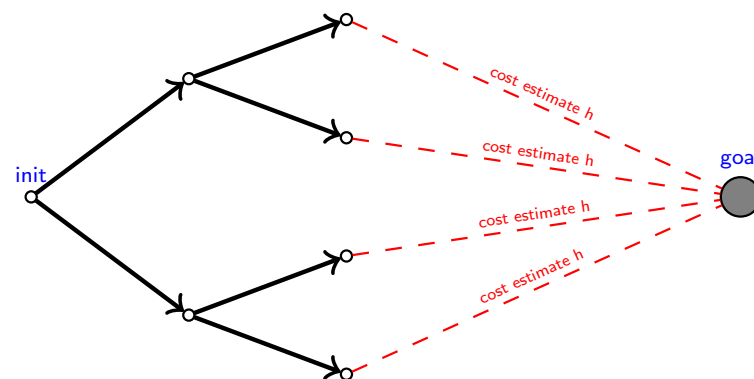


- **Facts  $P$ :**  $\{truck(x) \mid x \in \{A, B, C, D\}\} \cup \{pack(x) \mid x \in \{A, B, C, D, T\}\}$ .
- **Initial state  $I$ :**  $\{truck(A), pack(C)\}$ .
- **Goal  $G$ :**  $\{truck(A), pack(D)\}$ .
- **Actions  $A$ :** (Notated as “precondition  $\Rightarrow$  adds,  $\neg$  deletes”)
  - $drive(x, y)$ , where  $x, y$  have a road: “ $truck(x) \Rightarrow truck(y), \neg truck(x)$ ”.
  - $load(x)$ : “ $truck(x), pack(x) \Rightarrow pack(T), \neg pack(x)$ ”.
  - $unload(x)$ : “ $truck(x), pack(T) \Rightarrow pack(x), \neg pack(T)$ ”.





# Heuristic Search



→ Heuristic function  $h$  estimates the cost of an optimal path from a state  $s$  to the goal; search prefers to expand states  $s$  with small  $h(s)$ .

**Definition (Heuristic Function).** Let  $\Pi$  be a planning task with state space  $\Theta_{\Pi} = (S, L, c, T, I, S^G)$ . A **heuristic function**, short **heuristic**, for  $\Pi$  is a function  $h : S \mapsto \mathbb{R}_0^+ \cup \{\infty\}$ . Its value  $h(s)$  for a state  $s$  is referred to as the state's **heuristic value**, or  **$h$  value**.

**Definition (Remaining Cost,  $h^*$ ).** Let  $\Pi$  be a planning task with state space  $\Theta_{\Pi} = (S, L, c, T, I, S^G)$ . For a state  $s \in S$ , the state's **remaining cost** is the cost of an optimal plan for  $s$ , or  $\infty$  if there exists no plan for  $s$ . The **perfect heuristic** for  $\Pi$ , written  $h^*$ , assigns every  $s \in S$  its remaining cost as the heuristic value.

→ Heuristic functions  $h$  estimate remaining cost  $h^*$ .

## Properties of Individual Heuristic Functions

**Definition (Safe/Goal-Aware/Admissible/Consistent).** Let  $\Pi$  be a planning task with state space  $\Theta_{\Pi} = (S, L, c, T, I, S^G)$ , and let  $h$  be a heuristic for  $\Pi$ . The heuristic is called:

- **safe** if, for all  $s \in S$ ,  $h(s) = \infty$  implies  $h^*(s) = \infty$ ;
- **goal-aware** if  $h(s) = 0$  for all goal states  $s \in S^G$ ;
- **admissible** if  $h(s) \leq h^*(s)$  for all  $s \in S$ ;
- **consistent** if  $h(s) \leq h(s') + c(a)$  for all transitions  $s \xrightarrow{a} s'$ .

### → Relationships:

**Proposition.** Let  $\Pi$  be a planning task, and let  $h$  be a heuristic for  $\Pi$ . Then:

- If  $h$  is admissible, then  $h$  is goal-aware.
- If  $h$  is admissible, then  $h$  is safe.
- If  $h$  is consistent and goal-aware, then  $h$  is admissible.
- No other implications of this form hold.

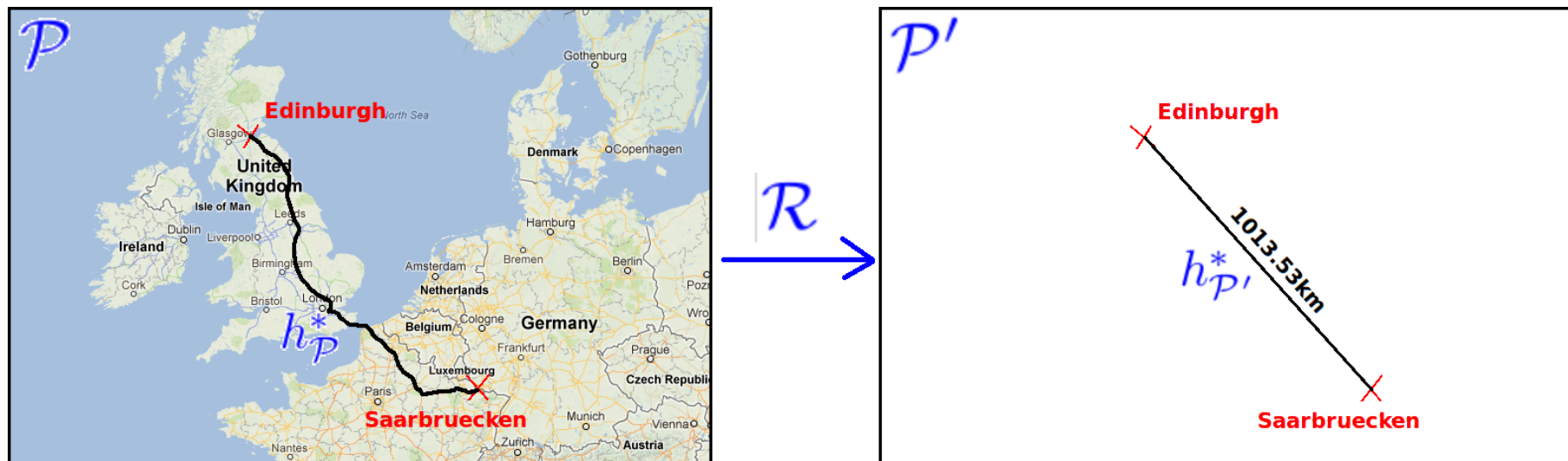
# Greedy Best-First Search and $A^*$

- Greedy best-first search explores states with lowest heuristic value  $h$ .
- $A^*$  explores states with lowest plan-cost estimate  $g + h$ .

**Greedy best-first search: Fast but not optimal  $\implies$  satisficing planning.**

**$A^*$ : Optimal for admissible  $h \implies$  optimal planning,  
with such  $h$ .**

# Relaxation in Route-Finding



- **Problem class  $\mathcal{P}$ :** Route finding.
- **Perfect heuristic  $h_{\mathcal{P}}^*$  for  $\mathcal{P}$ :** Length of a shortest route.
- **Simpler problem class  $\mathcal{P}'$ :** Route finding on an empty map.
- **Perfect heuristic  $h_{\mathcal{P}'}^*$  for  $\mathcal{P}'$ :** Straight-line distance.
- **Transformation  $\mathcal{R}$ :** Throw away the map.

## How the Delete Relaxation Changes the World

“When the world changes, its previous state remains true as well.”

Relaxed world: (before)



# How the Delete Relaxation Changes the World

“When the world changes, its previous state remains true as well.”

Relaxed world: (after)



# The Delete Relaxation

**Definition (Delete Relaxation).** Let  $\Pi = (P, A, I, G)$  be a planning task. The **delete-relaxation** of  $\Pi$  is the task  $\Pi^+ = (P, A^+, I, G)$  where  $A^+ = \{a^+ \mid a \in A\}$  with  $pre_{a^+} = pre_a$ ,  $add_{a^+} = add_a$ , and  $del_{a^+} = \emptyset$ .

→ In other words, the class of simpler problems  $\mathcal{P}'$  is the set of all STRIPS planning tasks with empty delete lists, and the relaxation mapping  $\mathcal{R}$  drops the delete lists.

**Definition (Relaxed Plan).** Let  $\Pi = (P, A, I, G)$  be a planning task, and let  $s$  be a state. A **relaxed plan** for  $s$  is a plan for  $(P, A, s, G)^+$ . A relaxed plan for  $I$  is called a **relaxed plan** for  $\Pi$ .

→ A relaxed plan for  $s$  is an action sequence that solves  $s$  when pretending that all delete lists are empty.

→ Also called **delete-relaxed plan**; “relaxation” is often used to mean “delete-relaxation” by default.

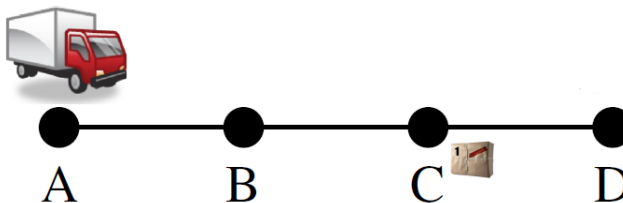
# A Relaxed Plan for “TSP” in Australia



- 1 **Initial state:**  $\{at(Sydney), visited(Sydney)\}$ .
- 2 **Apply**  $drive(Sydney, Brisbane)^+$ :  $\{at(Brisbane), visited(Brisbane), at(Sydney), visited(Sydney)\}$ .
- 3 **Apply**  $drive(Sydney, Adelaide)^+$ :  $\{at(Adelaide), visited(Adelaide), at(Brisbane), visited(Brisbane), at(Sydney), visited(Sydney)\}$ .
- 4 **Apply**  $drive(Adelaide, Perth)^+$ :  $\{at(Perth), visited(Perth), at(Adelaide), visited(Adelaide), at(Brisbane), visited(Brisbane), at(Sydney), visited(Sydney)\}$ .
- 5 **Apply**  $drive(Adelaide, Darwin)^+$ :  $\{at(Darwin), visited(Darwin), at(Perth), visited(Perth), at(Adelaide), visited(Adelaide), at(Brisbane), visited(Brisbane), at(Sydney), visited(Sydney)\}$ .



# A Relaxed Plan for “Logistics”



- **Facts  $P$ :**  $\{truck(x) \mid x \in \{A, B, C, D\}\} \cup \{pack(x) \mid x \in \{A, B, C, D, T\}\}$ .
- **Initial state  $I$ :**  $\{truck(A), pack(C)\}$ .
- **Goal  $G$ :**  $\{truck(A), pack(D)\}$ .
- **Relaxed actions  $A^+$ :** (Notated as “precondition  $\Rightarrow$  adds”)
  - $drive(x, y)^+$ : “ $truck(x) \Rightarrow truck(y)$ ”.
  - $load(x)^+$ : “ $truck(x), pack(x) \Rightarrow pack(T)$ ”.
  - $unload(x)^+$ : “ $truck(x), pack(T) \Rightarrow pack(x)$ ”.

## Relaxed plan:

$\langle drive(A, B)^+, drive(B, C)^+, load(C)^+, drive(C, D)^+, unload(D)^+ \rangle$

→ We don’t need to drive the truck back, because “it is still at  $A$ ”.

PlanEx<sup>+</sup>

**Definition (Relaxed Plan Existence Problem).** By **PlanEx<sup>+</sup>**, we denote the problem of deciding, given a planning task  $\Pi = (P, A, I, G)$ , whether or not there exists a **relaxed plan** for  $\Pi$ .

→ **This is easier than PlanEx for general STRIPS!**

**Proposition (PlanEx<sup>+</sup> is Easy).** *PlanEx<sup>+</sup> is a member of P.*

**Proof.** The following algorithm decides PlanEx<sup>+</sup>:

```
 $F := I$ 
while  $G \not\subseteq F$  do
     $F' := F \cup \bigcup_{a \in A: pre_a \subseteq F} add_a$ 
    (*) if  $F' = F$  then return “unsolvable” endif
     $F := F'$ 
endwhile
return “solvable”
```

The algorithm terminates after at most  $|P|$  iterations, and thus runs in polynomial time. Correctness: See post-handouts.

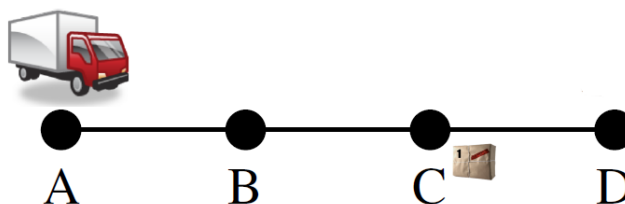
# Deciding PlanEx<sup>+</sup> in “TSP” in Australia (for reference)



## Iterations on $F$ :

- $\{at(Sydney), visited(Sydney)\}$
- $\cup \{at(Adelaide), visited(Adelaide), at(Brisbane), visited(Brisbane)\}$
- $\cup \{at(Darwin), visited(Darwin), at(Perth), visited(Perth)\}$

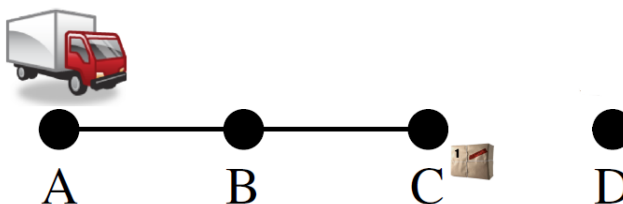
# Deciding PlanEx<sup>+</sup> in “Logistics”



## Iterations on $F$ :

- $\{truck(A), pack(C)\}$
- $\cup \{truck(B)\}$
- $\cup \{truck(C)\}$
- $\cup \{truck(D), pack(T)\}$
- $\cup \{pack(A), pack(B), pack(D)\}$

# Deciding PlanEx<sup>+</sup> in Unsolvable “Logistics”



## Iterations on $F$ :

- $\{truck(A), pack(C)\}$
- $\cup \{truck(B)\}$
- $\cup \{truck(C)\}$
- $\cup \{pack(T)\}$
- $\cup \{pack(A), pack(B)\}$
- $\cup \emptyset$

# PlanEx<sup>+</sup> Algorithm: Proof (for reference)

→ **Show: The algorithm returns “solvable” iff there exists a relaxed plan for  $\Pi$ .**

Denote by  $F_i$  the content of  $F$  after the  $i$ th iteration of the while-loop, and denote by  $A_i$  the set of actions  $a$  where  $pre_a \subseteq F_i$ .

All  $a \in A_0$  are applicable in  $I$ , all  $a \in A_1$  are applicable in  $appl(I, A_0^+)$ , and so forth. Thus  $F_i = appl(I, \langle A_0^+, \dots, A_{i-1}^+ \rangle)$ . (Within each  $A_j^+$ , we can sequence the actions in any order.)

**Direction “ $\Rightarrow$ ”:** If “solvable” is returned after iteration  $n$  then  $G \subseteq F_n = appl(I, \langle A_0^+, \dots, A_{n-1}^+ \rangle)$  so  $\langle A_0^+, \dots, A_{n-1}^+ \rangle$  can be sequenced to a relaxed plan which shows the claim.

**Direction “ $\Leftarrow$ ”:** Let  $\langle a_0^+, \dots, a_{n-1}^+ \rangle$  be a relaxed plan, hence  $G \subseteq appl(I, \langle a_0^+, \dots, a_{n-1}^+ \rangle)$ . Assume, for the moment, that we drop line (\*) from the algorithm. It is then easy to see that  $a_i \in A_i$  and  $appl(I, \langle a_0^+, \dots, a_{i-1}^+ \rangle) \subseteq F_i$ , for all  $i$ . We get  $G \subseteq appl(I, \langle a_0^+, \dots, a_{n-1}^+ \rangle) \subseteq F_n$ , and the algorithm returns “solvable” as desired.

Assume to the contrary of the claim that, in an iteration  $i < n$ , (\*) fires. Then  $G \not\subseteq F_i$  and  $F_i = F_{i+1}$ . But, then,  $F_i = F_j$  for all  $j > i$ , and we get  $G \not\subseteq F_n$  in contradiction.

# Questionnaire

## Question!

How does ignoring delete lists simplify Sokoban?

(A): You will never “lock yourself in”.

(B): Free positions remain free.

(C): You can walk through walls.

(D): A single action can push 2 stones at once.

→ (A): Yes, because of (B).

→ (B): Yes, when we move a stone into a free space, that space is still free afterwards.

→ (C): No, we don't get any new moves in the relaxation.

→ (D): It depends on the model. Within the relaxed problem, it may happen that two stones are in the same position, so in principle we can push them both. However, without distinguishing stone names, it is impossible to separate them again, so the two stones in fact “become” (behave in all relevant ways exactly like) a single stone.

# The $h^+$ Heuristic

→ **PlanEx<sup>+</sup> is not actually what we're looking for. PlanEx<sup>+</sup> = relaxed plan existence; we want relaxed plan length  $h^* \circ \mathcal{R}$ .**

**Definition (Optimal Relaxed Plan).** Let  $\Pi = (P, A, I, G)$  be a planning task, and let  $s$  be a state. An **optimal relaxed plan** for  $s$  is an optimal plan for  $(P, A, s, G)^+$ .

Here's what we're looking for:

**Definition ( $h^+$ ).** Let  $\Pi = (P, A, I, G)$  be a planning task with states  $S$ . The **ideal delete-relaxation heuristic  $h^+$**  for  $\Pi$  is the function  $h^+ : S \mapsto \mathbb{N}_0 \cup \{\infty\}$  where  $h^+(s)$  is the length of an optimal relaxed plan for  $s$  if a relaxed plan for  $s$  exists, and  $h^+(s) = \infty$  otherwise.

→ In other words,  $h^+ = h^* \circ \mathcal{R}$ , cf. previous slide.



## $h^+$ is Admissible

**Lemma.** Let  $\Pi = (P, A, I, G)$  be a planning task, and let  $s$  be a state. If  $\langle a_1, \dots, a_n \rangle$  is a plan for  $(P, A, s, G)$ , then  $\langle a_1^+, \dots, a_n^+ \rangle$  is a plan for  $(P, A, s, G)^+$ .

**Proof Sketch. (for reference)** Show by induction over  $0 \leq i \leq n$  that  $appl(s, \langle a_1, \dots, a_i \rangle) \subseteq appl(s, \langle a_1^+, \dots, a_i^+ \rangle)$ .

→ "If we ignore deletes, the states along the plan can only get bigger."

**Theorem.**  $h^+$  is Admissible.

**Proof. (for reference)** Let  $\Pi = (P, A, I, G)$  be a planning task with states  $S$ , and let  $s \in S$ .  $h^+(s)$  is defined as optimal plan length in  $(P, A, s, G)^+$ . With the above lemma, any plan for  $(P, A, s, G)$  also constitutes a plan for  $(P, A, s, G)^+$ . Thus optimal plan length in  $(P, A, s, G)^+$  cannot be longer than that in  $(P, A, s, G)$ , and the claim follows.

# $h^+$ in “TSP” in Australia



## Planning vs. Relaxed Planning:

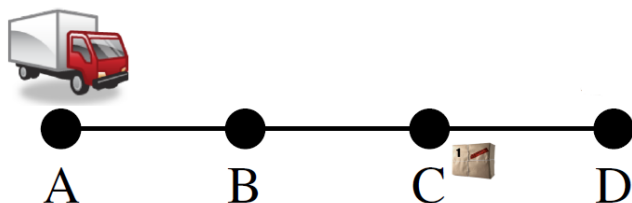
- **Optimal plan:**  $\langle \text{drive}(\text{Sydney}, \text{Brisbane}), \text{drive}(\text{Brisbane}, \text{Sydney}), \text{drive}(\text{Sydney}, \text{Adelaide}), \text{drive}(\text{Adelaide}, \text{Perth}), \text{drive}(\text{Perth}, \text{Adelaide}), \text{drive}(\text{Adelaide}, \text{Darwin}), \text{drive}(\text{Darwin}, \text{Adelaide}), \text{drive}(\text{Adelaide}, \text{Sydney}) \rangle$ .
- **Optimal relaxed plan:**  $\langle \text{drive}(\text{Sydney}, \text{Brisbane}), \text{drive}(\text{Sydney}, \text{Adelaide}), \text{drive}(\text{Adelaide}, \text{Perth}), \text{drive}(\text{Adelaide}, \text{Darwin}) \rangle$ .
- $h^*(I) = 8; h^+(I) = 4$ .

## How to Relax During Search: Overview

**Attention!** Search uses the real (un-relaxed)  $\Pi$ . The relaxation is applied (e.g., in  $h^+$ , the simplified actions are used) only within the call to  $h(s)$ !!!

- Two common student mistake is to instead apply the relaxation once to the whole problem, then doing the whole search “within the relaxation”.
- The next slide illustrates the correct search process in detail.

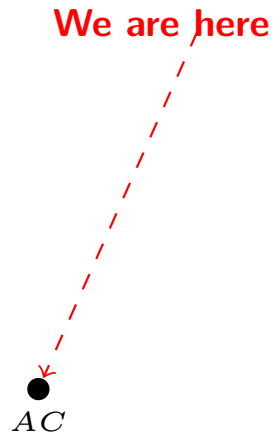
# How to Relax During Search: Ignoring Deletes



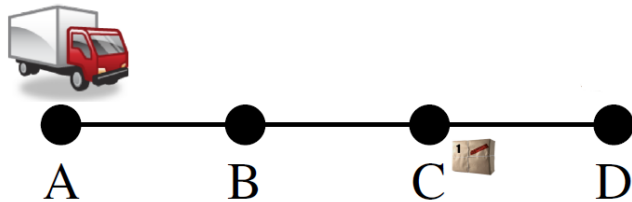
## Real problem:

- Initial state  $I$ :  $AC$ ; goal  $G$ :  $AD$ .
- Actions  $A$ :  $pre$ ,  $add$ ,  $del$ .
- $drXY$ ,  $loX$ ,  $ulX$ .

**Greedy best-first search:**  
(tie-breaking: alphabetic)



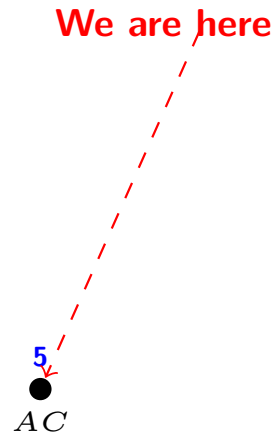
# How to Relax During Search: Ignoring Deletes



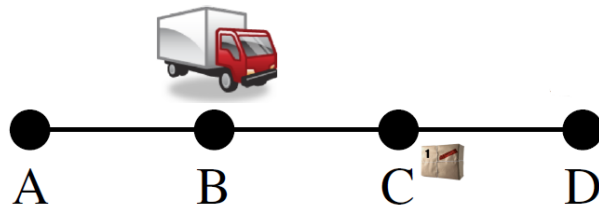
## Relaxed problem:

- State  $s$ :  $AC$ ; goal  $G$ :  $AD$ .
- Actions  $A$ : *pre*, *add*.
- $h^+(s) = 5$ : e.g.  $\langle drAB, drBC, drCD, loC, ulD \rangle$ .

**Greedy best-first search:**  
(tie-breaking: alphabetic)



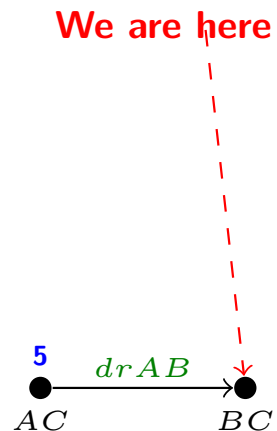
# How to Relax During Search: Ignoring Deletes



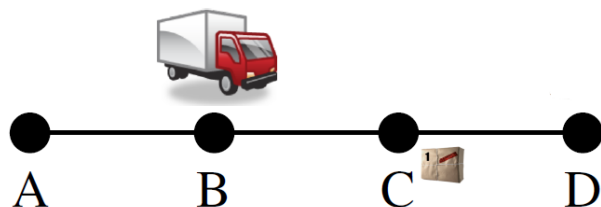
## Real problem:

- State  $s$ :  $BC$ ; goal  $G$ :  $AD$ .
- Actions  $A$ :  $pre$ ,  $add$ ,  $del$ .
- $AC \xrightarrow{drAB} BC$ .

**Greedy best-first search:**  
(tie-breaking: alphabetic)



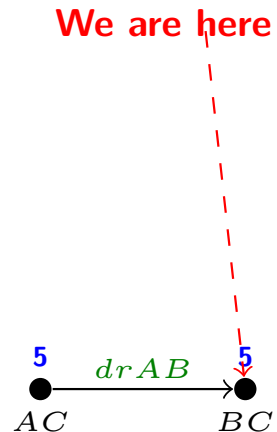
# How to Relax During Search: Ignoring Deletes



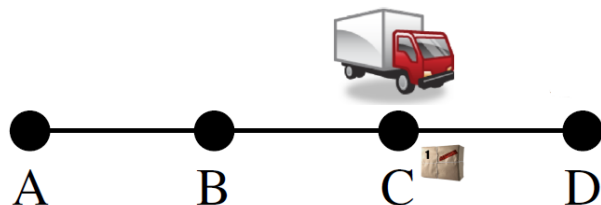
## Relaxed problem:

- State  $s$ :  $BC$ ; goal  $G$ :  $AD$ .
- Actions  $A$ : *pre*, *add*.
- $h^+(s) = 5$ : e.g.  $\langle drBA, drBC, drCD, loC, ulD \rangle$ .

**Greedy best-first search:**  
(tie-breaking: alphabetic)



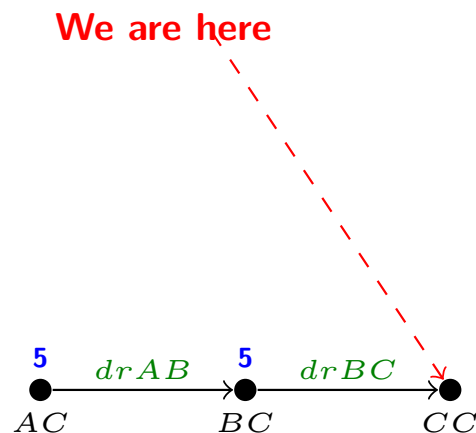
# How to Relax During Search: Ignoring Deletes



## Real problem:

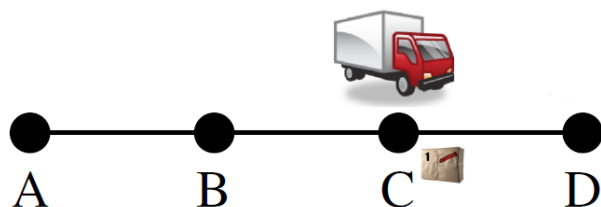
- State  $s$ :  $CC$ ; goal  $G$ :  $AD$ .
- Actions  $A$ :  $pre$ ,  $add$ ,  $del$ .
- $BC \xrightarrow{dr^{BC}} CC$ .

**Greedy best-first search:**  
(tie-breaking: alphabetic)





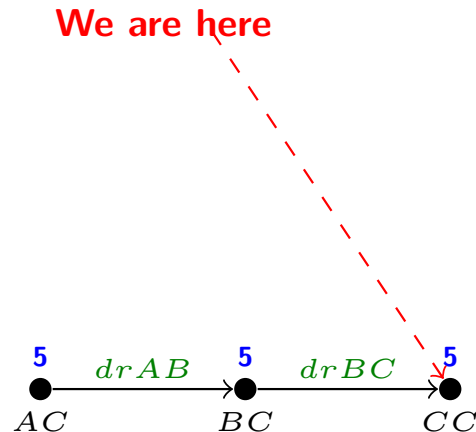
# How to Relax During Search: Ignoring Deletes



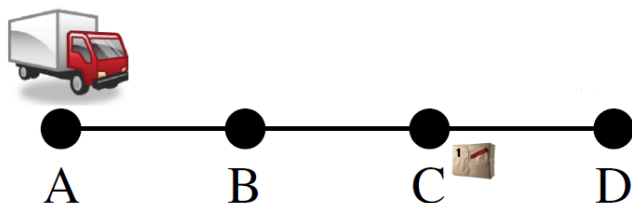
## Relaxed problem:

- State  $s$ :  $CC$ ; goal  $G$ :  $AD$ .
- Actions  $A$ :  $pre, add$ .
- $h^+(s) = 5$ : e.g.  $\langle drCB, drBA, drCD, loC, ulD \rangle$ .

**Greedy best-first search:**  
(tie-breaking: alphabetic)



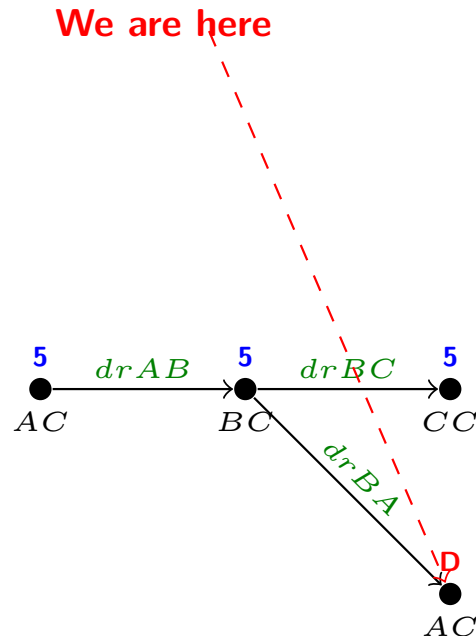
# How to Relax During Search: Ignoring Deletes



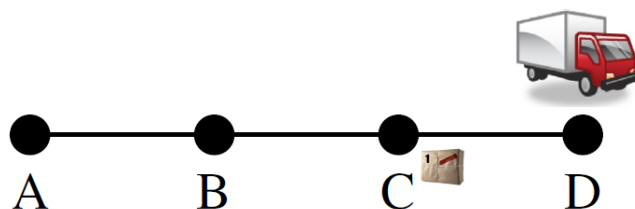
## Real problem:

- State  $s$ :  $AC$ ; goal  $G$ :  $AD$ .
- Actions  $A$ :  $pre$ ,  $add$ ,  $del$ .
- **Duplicate state, prune.**

**Greedy best-first search:**  
(tie-breaking: alphabetic)



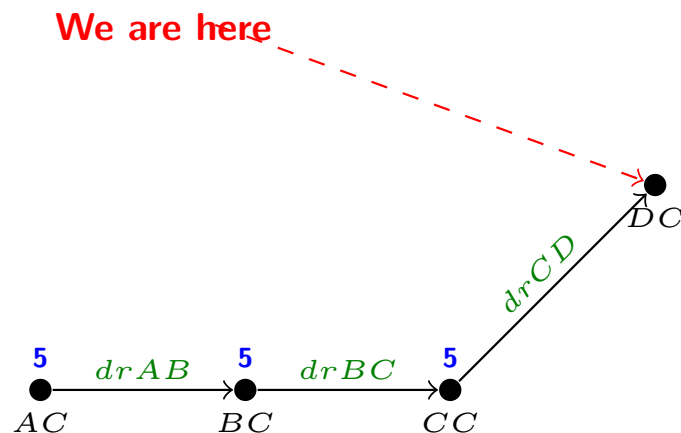
# How to Relax During Search: Ignoring Deletes



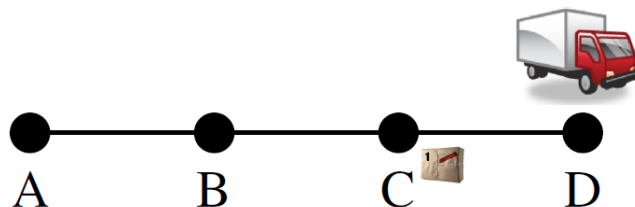
## Real problem:

- State  $s$ :  $DC$ ; goal  $G$ :  $AD$ .
- Actions  $A$ :  $pre$ ,  $add$ ,  $del$ .
- $CC \xrightarrow{dr^{CD}} DC$ .

**Greedy best-first search:**  
(tie-breaking: alphabetic)



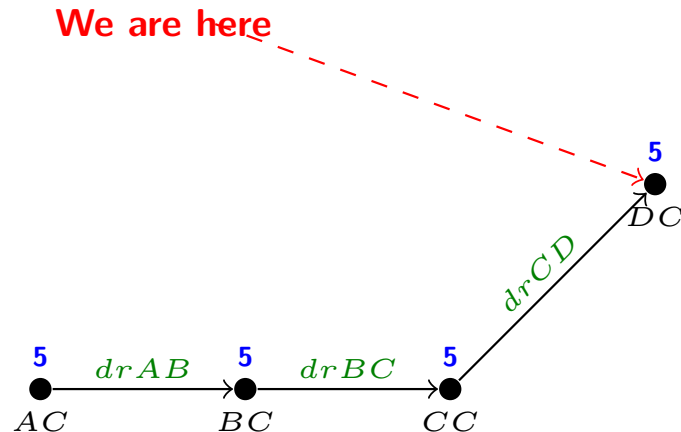
# How to Relax During Search: Ignoring Deletes



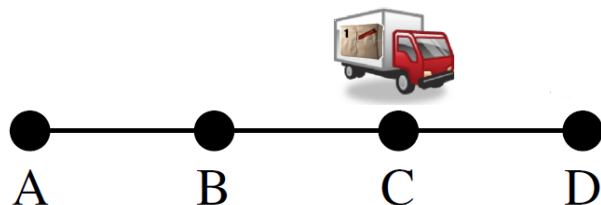
## Relaxed problem:

- State  $s$ :  $DC$ ; goal  $G$ :  $AD$ .
- Actions  $A$ : *pre*, *add*.
- $h^+(s) = 5$ : e.g.  $\langle drDC, drCB, drBA, loC, ulD \rangle$ .

**Greedy best-first search:**  
(tie-breaking: alphabetic)



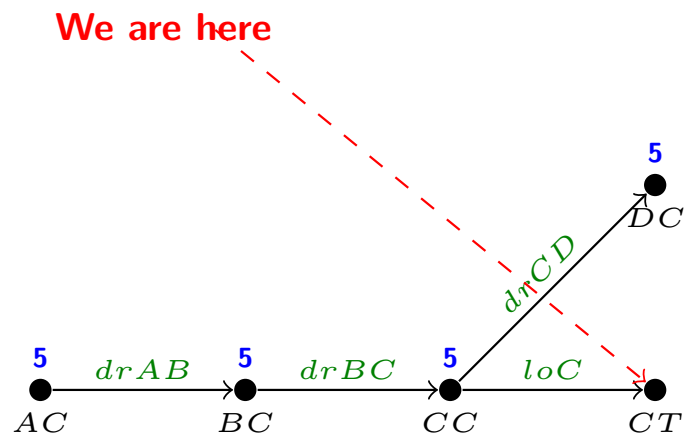
# How to Relax During Search: Ignoring Deletes



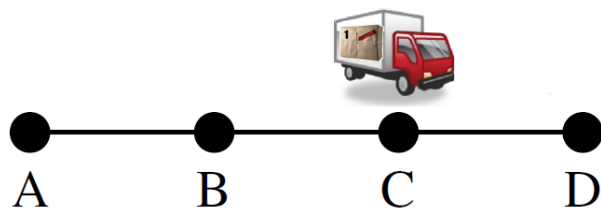
## Real problem:

- State  $s$ :  $CT$ ; goal  $G$ :  $AD$ .
- Actions  $A$ :  $pre$ ,  $add$ ,  $del$ .
- $CC \xrightarrow{loC} CT$ .

**Greedy best-first search:**  
(tie-breaking: alphabetic)



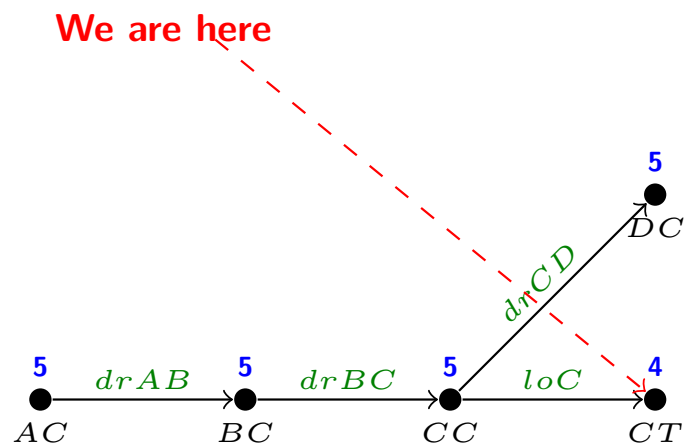
# How to Relax During Search: Ignoring Deletes



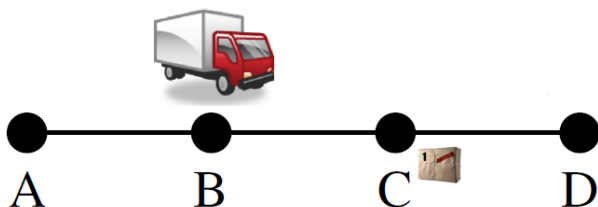
## Relaxed problem:

- State  $s$ :  $CT$ ; goal  $G$ :  $AD$ .
- Actions  $A$ :  $pre, add$ .
- $h^+(s) = 4$ : e.g.  $\langle drCB, drBA, drCD, ulD \rangle$ .

Greedy best-first search:  
(tie-breaking: alphabetic)



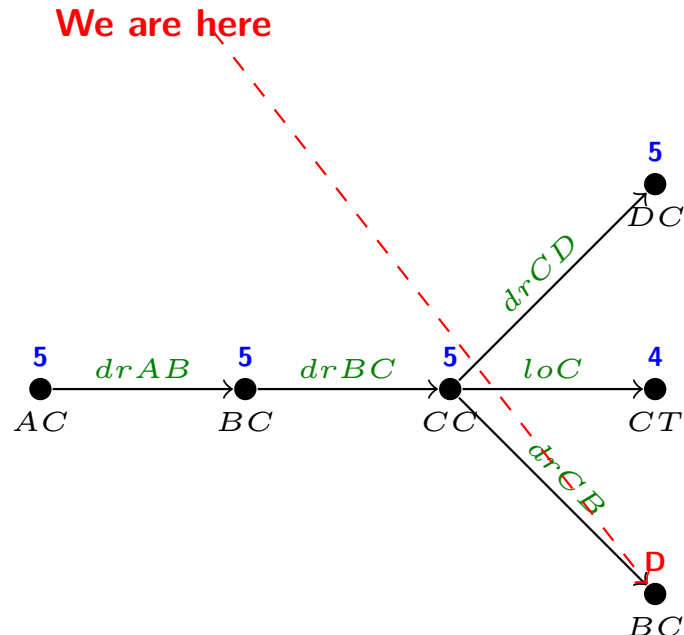
# How to Relax During Search: Ignoring Deletes



## Real problem:

- State  $s$ :  $BC$ ; goal  $G$ :  $AD$ .
- Actions  $A$ :  $pre$ ,  $add$ ,  $del$ .
- **Duplicate state, prune.**

Greedy best-first search:  
(tie-breaking: alphabetic)







## Approximating $h^+$ : $h^{\text{FF}}$

**Theorem.**  $\text{PlanLen}^+$  is **NP-complete**.

→ We can't compute our heuristic  $h^+$  efficiently. So we approximate it instead.

**Definition ( $h^{\text{FF}}$ ).** Let  $\Pi = (P, A, I, G)$  be a planning task with states  $S$ . A **relaxed plan heuristic**  $h^{\text{FF}}$  for  $\Pi$  is a function  $h^{\text{FF}} : S \mapsto \mathbb{N}_0 \cup \{\infty\}$  returning **the length of some, not necessarily optimal, relaxed plan for  $s$**  if a relaxed plan for  $s$  exists, and returning  $h^{\text{FF}}(s) = \infty$  otherwise.

**Notes:**

- $h^{\text{FF}} \geq h^+$ , i.e.,  $h^{\text{FF}}$  never under-estimates  $h^+$ .
- **We may have  $h^{\text{FF}} > h^*$ , i.e.,  $h^{\text{FF}}$  is not admissible!** Thus  $h^{\text{FF}}$  can be used for satisficing planning only, not for optimal planning.

**Observe:**  $h^{\text{FF}}$  as per this definition is not unique. How do we find “some, not necessarily optimal, relaxed plan for  $(P, A, s, G)$ ”?

→ In what follows, we consider the following algorithm computing relaxed plans, and therewith (one variant of)  $h^{\text{FF}}$ :

- ① Chain **forward** to build a **relaxed planning graph (RPG)**.
- ② Chain **backward** to extract a relaxed plan from the RPG.

# Computing $h^{\text{FF}}$ : Relaxed Planning Graphs (RPG)

```
 $F_0 := s, t := 0$   
while  $G \not\subseteq F_t$  do  
     $A_t := \{a \in A \mid \text{pre}_a \subseteq F_t\}$   
     $F_{t+1} := F_t \cup \bigcup_{a \in A_t} \text{add}_a$   
    if  $F_{t+1} = F_t$  then stop endif  
     $t := t + 1$   
endwhile
```

→ **Does this look familiar to you?** Could. It's the same algorithm we used to decide PlanEx<sup>+</sup> (slide 35).

**“Logistics” example:** (similar to slide 37).

**Are we done?** “ $\langle A_0^+, \dots, A_{n-1}^+ \rangle$  can be sequenced to a relaxed plan”. Could use this as the basis of  $h^{\text{FF}}$ .

→ **But this would overestimate vastly!**

In “Logistics”,  $\sum_{i=0}^{n-1} |A_i| = 11 > 8 = h^*$ . And now imagine there are 100 packages only one of which needs to be transported ...

# Computing $h^{\text{FF}}$ : Extracting a Relaxed Plan

Information from the RPG: (min over an empty set is  $\infty$ )

- For  $p \in P$ :  $level(\mathbf{p}) := \min\{\mathbf{t} \mid \mathbf{p} \in \mathbf{F}_t\}$ .
- For  $a \in A$ :  $level(\mathbf{a}) := \min\{\mathbf{t} \mid \mathbf{a} \in \mathbf{A}_t\}$ .

```
 $M := \max\{level(p) \mid p \in G\}$ 
If  $M = \infty$  then  $h^{\text{FF}}(s) := \infty$ ; stop endif
for  $t := 0, \dots, M$  do
   $G_t := \{g \in G \mid level(g) = t\}$ 
endfor
for  $t := M, \dots, 1$  do
  for all  $g \in G_t$  do
    select  $a$ ,  $level(a) = t - 1$ ,  $g \in add_a$ 
    for all  $p \in pre_a$  do  $G_{level(p)} := G_{level(p)} \cup \{p\}$  endfor
  endfor
endfor
 $h^{\text{FF}}(s) := \text{number of selected actions}$ 
```

# Computing $h^{\text{FF}}$ in “TSP” in Australia



## RPG:

- $F_0 = \{at(Sydney), visited(Sydney)\}.$
- $A_0 = \{drive(Sydney, Adelaide), drive(Sydney, Brisbane)\}.$
- $F_1 = F_0 \cup \{at(Adelaide), visited(Adelaide), at(Brisbane), visited(Brisbane)\}.$
- $A_1 = A_0 \cup \{drive(Adelaide, Darwin), drive(Adelaide, Perth), drive(Adelaide, Sydney), drive(Brisbane, Sydney)\}.$
- $F_2 = F_1 \cup \{at(Darwin), visited(Darwin), at(Perth), visited(Perth)\}.$

# Computing $h^{\text{FF}}$ in “TSP” in Australia



## Inserting the goals:

- $F_0$ : *at(Sydney)*, *visited(Sydney)*.
- $A_0$ : *drive(Sydney, Adelaide)*, *drive(Sydney, Brisbane)*.
- $F_1$ : *at(Adelaide)*, *visited(Adelaide)*, *at(Brisbane)*, *visited(Brisbane)*.
- $A_1$ : *drive(Adelaide, Darwin)*, *drive(Adelaide, Perth)*, *drive(Adelaide, Sydney)*, *drive(Brisbane, Sydney)*.
- $F_2$ : *at(Darwin)*, *visited(Darwin)*, *at(Perth)*, *visited(Perth)*.

# Computing $h^{\text{FF}}$ in “TSP” in Australia



## Supporting the goals at $t = 2$ :

- $F_0$ : *at(Sydney)*, *visited(Sydney)*.
- $A_0$ : *drive(Sydney, Adelaide)*, *drive(Sydney, Brisbane)*.
- $F_1$ : *at(Adelaide)*, *visited(Adelaide)*, *at(Brisbane)*, *visited(Brisbane)*.
- $A_1$ : *drive(Adelaide, Darwin)*, *drive(Adelaide, Perth)*, *drive(Adelaide, Sydney)*, *drive(Brisbane, Sydney)*.
- $F_2$ : *at(Darwin)*, *visited(Darwin)*, *at(Perth)*, *visited(Perth)*.

# Computing $h^{\text{FF}}$ in “TSP” in Australia



## Supporting the goals at $t = 1$ :

- $F_0$ : *at(Sydney), visited(Sydney)*.
- $A_0$ : *drive(Sydney, Adelaide), drive(Sydney, Brisbane)*.
- $F_1$ : *at(Adelaide), visited(Adelaide), at(Brisbane), visited(Brisbane)*.
- $A_1$ : *drive(Adelaide, Darwin), drive(Adelaide, Perth), drive(Adelaide, Sydney), drive(Brisbane, Sydney)*.
- $F_2$ : *at(Darwin), visited(Darwin), at(Perth), visited(Perth)*.

# Summary

- **Planning** is a form of general problem solving: develop solvers that perform well across a large class of problems.
- STRIPS is the simplest possible, while reasonably expressive, language for our purposes. It uses Boolean variables (facts), and defines actions in terms of precondition, add list, and delete list.
- We will consider Greedy Best-First Search and  $A^*$  as heuristic search algorithms.
- Heuristic search on classical search problems relies on a function  $h$  mapping states  $s$  to an estimate  $h(s)$  of their goal distance. Such functions  $h$  are derived by solving **relaxed problems**.
- In planning, the relaxed problems are generated and solved automatically.
- The **delete relaxation** consists in dropping the deletes from STRIPS planning tasks. A **relaxed plan** is a plan for such a relaxed task.  $h^+(s)$  is the length of an optimal relaxed plan for state  $s$ .